

Vector Policy Optimization: Training for Diversity Improves Test-Time Search

Ryan Bahlous-Boldi^{1,2} Isha Puri¹ Idan Shenfeld^{1,2} Akarsh Kumar¹ Mehul Damani¹
Sebastian Risi⁴ Omar Khattab¹ Zhang-Wei Hong^{1,2,3} Pulkit Agrawal^{1,2}

¹MIT ²Improbable AI Lab ³MIT-IBM Computing Research Lab ⁴Sakana AI

Abstract

Language models must now generalize out of the box to novel environments and work inside inference-scaling search procedures, such as AlphaEvolve, that select rollouts with a variety of task-specific reward functions. Unfortunately, the standard paradigm of LLM post-training optimizes a pre-specified scalar reward, often leading current LLMs to produce low-entropy response distributions and thus to struggle at displaying the diversity that inference-time search will require. We propose **Vector Policy Optimization (VPO)**, an RL algorithm that explicitly trains policies to anticipate diverse downstream reward functions and to produce diverse solutions. VPO exploits that rewards are often vector-valued in practice, like per-test-case correctness in code generation or, say, multiple different user personas or reward models. VPO is essentially a drop-in replacement for the GRPO advantage estimator, but it trains the LLM to output a *set* of solutions where individual solutions specialize to different trade-offs in the vector reward space. Across four tasks, VPO matches or beats the strongest scalar RL baselines on test-time search (e.g. pass@ k and best@ k), with the gap widening as the search budget grows. For evolutionary search, VPO models unlock problems that GRPO models cannot solve at all. As test-time search becomes more standardized, optimizing for diversity may need to become the default post-training objective.

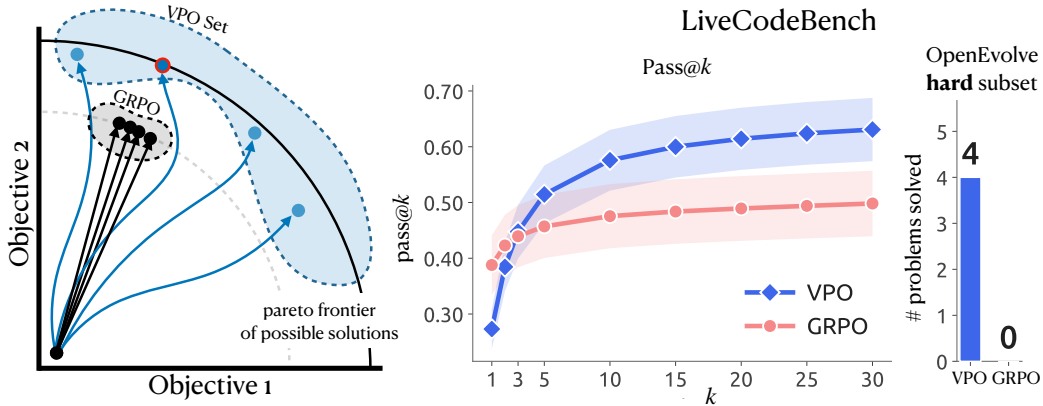


Figure 1: **Vector Policy Optimization (VPO)**. When maximizing a scalar, GRPO sends all solutions to the same, potentially sub-optimal, solution. VPO simultaneously optimizes across different reward weightings, increasing the chance of finding a superior target solution. As a result, on LiveCodeBench, for example, VPO results in better test-time search performance, whether using pass@ k or even complex evolutionary test-time search like AlphaEvolve.

1 Introduction

Exploration is a core principle in reinforcement learning (RL). For learning to keep progressing, an agent must maintain diverse behavior, trying different strategies rather than committing to one prematurely. Balancing exploration and exploitation [Ladosz et al., 2022] is well studied in both classical RL [Sutton and Barto, 1998, Bellemare et al., 2016, Pathak et al., 2017] and in the context of modern foundation models [Chen et al., 2025, Setlur et al., 2025, Qu et al., 2026, Hong et al., 2024]. This trade-off remains a largely unsolved problem particularly for foundation models.

In many AI systems, the network is only one component of a larger pipeline. Especially for hard problems, language models are typically wrapped in some form of search, ranging from simple rejection sampling with a verifier [Cobbe et al., 2021, Brown et al., 2024] to complex evolutionary methods like AlphaEvolve [Novikov et al., 2025, Lange et al., 2025]. In these settings, test-time search handles exploitation, hinting that training should focus on providing the search with a *rich and diverse pool of solutions* to select from. However, existing RL post-training methods are poorly suited for this kind of diversity preservation. Policy gradient methods like GRPO [Shao et al., 2024] drive the policy toward a narrow set of high-probability responses [Wu and Choi, 2025, Yue et al., 2025]. After training, the diversity required for effective test-time search disappears, as additional samples become near-duplicates [GX-Chen et al., 2025, Kirk et al., 2024, Karouzos et al., 2026].

In this work, we propose a shift in perspective. Rather than asking a single training algorithm to handle both exploration and exploitation, we separate the two responsibilities entirely by assuming a future test-time exploitation stage. **In this setting, the role of RL post-training should not be to converge on a single best response, but to maximize the diversity of a set of competent solutions.** Later, during test-time, the search method will select among them.

To train a policy that produces diverse yet competent solutions, we exploit the fact that, in many realistic tasks, rewards can be naturally decomposed into a vector of components: per-test-case correctness for code generation, per-criterion ratings for RLHF, or per-sub-question success in multi-hop reasoning. This decomposition provides a natural axis for diversity. Rather than collapsing these components into a single scalar and optimizing toward one peak, we can encourage the model to produce solutions that excel along different reward dimensions, covering the Pareto frontier rather than converging to a single point on it [Rojers et al., 2013]. We term this optimization scheme **Vector Policy Optimization (VPO)**.

Concretely, VPO combines multi-answer generation [Puri et al., 2026] with stochastic reward scalarizations, training the model to produce sets of candidates that span the Pareto frontier rather than collapsing onto a single point. Together, these mechanisms maintain a richer candidate distribution so that test-time search can extract increasingly better solutions as the sample budget grows.

We evaluate VPO across four diverse settings spanning multi-hop question answering, logic reasoning, navigation, tool use and coding. Empirically, VPO matches or beats the strongest scalar baselines on test-time best@ k across our four benchmarks, with the gap widening as the candidate budget grows. The advantage holds at scale: on LiveCodeBench, a VPO-trained Qwen2.5-Coder-7B-Instruct improves both pass@ k and best@ k over a matched-compute GRPO checkpoint, and inside the OpenEvolve search loop unlocks problems that GRPO cannot solve at any candidate budget (Fig. 1). Our main contributions are:

- We argue that in AI systems where test-time search is available, RL post-training should focus exclusively on producing diverse, competent solutions, leaving exploitation to search.
- We show that the vector-valued structure of rewards in many practical settings provides a natural mechanism for achieving this diversity, by training the model to cover the Pareto frontier of the different objectives.
- We propose Vector Policy Optimization (VPO), a concrete instantiation of this idea that combines randomized reward scalarizations with the in-context capabilities of language models to generate diverse candidate sets within a single rollout.

2 What kind of diversity are we after?

Motivation A downstream search procedure only benefits from diversity if the candidates differ along specific axes the search requires. Surface-level variation, semantic diversity, or noisy sampling

are not enough. Search needs a pool of candidates that realizes different high-quality trade-offs between the objectives underlying the task. This becomes important once language models are deployed inside search-augmented systems. At inference time, the model is no longer evaluated one response at a time. Instead, the system generates many candidate solutions and selects among them. In this regime, committing the entire policy to a single trade-off is unnecessarily restrictive. The goal is no longer to produce one response that is optimal under a single fixed objective, but to produce a *set* of responses that spans multiple plausible trade-offs, so downstream search can choose among them.

We call this property *reward diversity*. A reward-diverse candidate pool contains solutions that are each optimal under different weightings of the underlying reward components. Intuitively, the policy remains deliberately non-committal: instead of collapsing onto a single mode, it preserves multiple strategies that perform well under different preferences.

Setting. Let x denote a prompt and y a response sampled from policy $\pi_\theta(\cdot \mid x)$. In many practical tasks, the reward signal decomposes naturally into d components, $r(x, y) = [r_1(x, y), \dots, r_d(x, y)] \in \mathbb{R}^d$, where each r_i captures a distinct aspect of response quality. For example, r_i may be per-test-case correctness in code generation [Chen et al., 2021], per-criterion preference scores in RLHF [Wang et al., 2024], per-hop correctness in multi-hop reasoning [Trivedi et al., 2022], or per-tool-call structural and content scores in agentic tasks [Qian et al., 2025]. Any weighting $w \in \Delta^{d-1}$ over the simplex induces a scalar objective $w^\top r(x, y)$; standard post-training fixes a single w^* and trains the policy to maximize $\mathbb{E}_{y \sim \pi_\theta(\cdot \mid x)} [w^{*\top} r(x, y)]$.

Under the standard single-response framing, this is sensible: if the policy emits only one answer, and evaluation uses a known weighting w^* , then directly optimizing w^* is the correct objective. The situation changes once inference-time search is introduced. Search benefits precisely from candidates that realize different trade-offs. A policy trained only under w^* has no incentive to preserve such alternatives, and policy-gradient training instead concentrates probability mass onto whichever strategy currently maximizes the scalarized reward. Additional samples then become increasingly redundant.

Crucially, preserving these alternatives is useful even when the deployment objective w^* is known in advance. The reason is that search operates over *sets* of candidates rather than individual responses. A candidate pool that spans multiple reward trade-offs gives the search procedure more opportunities to discover high-performing solutions under w^* itself.

Under scalar training, optimization commits aggressively toward whichever responses currently score highest under w^* . Alternative strategies that sacrifice one component in exchange for another are suppressed early, even if they would eventually lead to stronger solutions under the same final objective. As training progresses, the candidate distribution collapses, and additional samples become increasingly redundant. Reward diversity counteracts this collapse by preserving solutions that are optimal under different regions of the reward simplex. Many of these solutions may appear locally suboptimal under w^* , yet still contain partial reasoning patterns, decompositions, or strategies that ultimately yield higher-scoring outcomes under w^* itself. In this sense, reward diversity functions as a structured form of exploration: instead of committing prematurely to a single trade-off, the policy maintains a population of competent alternatives long enough for search to exploit them.

This perspective connects naturally to work in multi-objective RL [Rojers et al., 2013, Hayes et al., 2022] and to lexicase selection in evolutionary computation [Spector, 2012, Spector et al., 2024, La Cava et al., 2019, Ni et al., 2024]. Both preserve solutions that are optimal under different subsets or weightings of objectives rather than collapsing all criteria into a single aggregate score. Importantly, however, our goal is different from classical multi-objective optimization. We do not seek a policy conditioned on user-specified preferences, nor do we assume the deployment objective is unknown. Our goal remains performance under a fixed weighting w^* . The key difference is that in a search-augmented regime, the best way to optimize w^* may be to train a policy that maintains reward-diverse candidate sets rather than immediately collapsing onto a single optimum.

3 Method: Vector Policy Optimization

In Section 2, we argued that the right RL target is reward-diverse sets, or collections of candidates that are each optimal under some weighting of the reward components. This section describes our

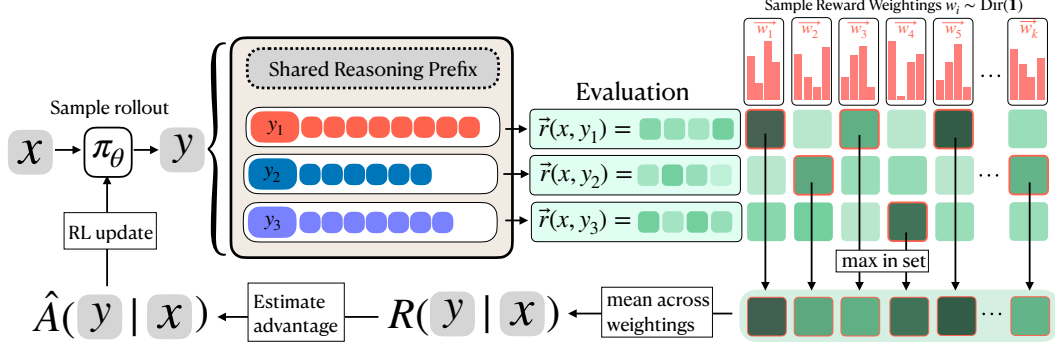


Figure 2: Outline of Vector Policy Optimization. Given a prompt x , a model π_θ outputs m answers in a single autoregressive chain. Each answer y_i is evaluated on multiple objectives and receives a score vector $[r_1, r_2, \dots, r_n]$. We repeatedly sample weighting vectors $w_i \sim \text{Dir}(\mathbf{1})$, and evaluate the mean best-of- m under each weighting w in the set of answers. This set level reward is treated as the reward for the entire completion $\pi_\theta(x)$.

proposed algorithm, **Vector Policy Optimization (VPO)**, which trains a policy to produce such sets. VPO has two key components. First, we train a model to generate multiple candidate completions per prompt within a single autoregressive rollout. Second, we replace a fixed reward weighting with a distribution over weights, so the model is incentivized to span its candidates across different trade-offs between the sub-objectives.

Together, these components define a set-level objective that rewards the model for producing diverse, high-quality solutions. We describe each component in detail below.

3.1 Multi-Answer Chains as In-Context Exploration

Following Puri et al. [2026], we train a language model to produce a set of m candidate completions $S = \{y_1, \dots, y_m\}$ within a single rollout. The completions are emitted sequentially, separated by a delimiter token, so when generating y_i , the prefix already contains $y_1, \dots, y_{i-1}$.

This fundamentally changes the nature of exploration. Under standard independent sampling, diversity arises only from stochastic decoding applied to a fixed conditional distribution, producing small variations around whichever mode the policy has concentrated on. In multi-answer rollouts, each new candidate can attend to the ones already emitted, giving the model the capacity to recognize which regions of the solution space are covered and steer subsequent candidates towards different ones. Diversity becomes an explicit, in-context mechanism rather than a byproduct of sampling noise.

Importantly, this mechanism provides the *capacity* for diversity, but not a strong *incentive*¹. Without an appropriate training signal, the model will still collapse to producing near-identical answers. We confirm this empirically in Section 5: Multi-RLVR, which combines multi-answer rollouts with a fixed scalar reward, produced sets whose reward-diversity collapses early in training. The objective defined in the next section supplies this missing incentive by rewarding sets whose elements specialize to different reward trade-offs.

3.2 Set-Level Optimization via Stochastic Scalarization

To train a policy to output diverse sets, we replace the fixed scalarization prevalent in RL post training with a distribution over scalarizations. For each rollout, we sample weights $w \sim \text{Dir}(\alpha)$, where $\alpha \in \mathbb{R}_{>0}^d$ defines a distribution over the simplex Δ^{d-1} . We use $\alpha = 1$ throughout, which results in uniform distribution over the simplex (i.e. the set of d -dimensional vectors that sum to 1). We evaluate a set $S = \{y_1, \dots, y_m\}$ under each sampled scalarization by selecting the best-performing element:

$$R(S) = \mathbb{E}_{w \sim \text{Dir}(\alpha)} \left[\max_{y \in S} w^\top r(x, y) \right] \quad (1)$$

¹In practice, Puri et al. [2026] de-duplicate their responses before reward calculation, which adds an incentive against direct duplicates, but the underlying objective still doesn't reward genuine specialization

This objective directly rewards coverage of the reward space: different elements in S are optimal under different samples of w . A set that collapses to identical responses performs well under a narrow region of the simplex, while a set that spans multiple trade-offs performs well across many scalarizations. This objective can be seen as directly optimizing expected best-of- N over sampled w .

The two components of VPO are complementary: multi-answer generation alone does not incentivize diversity, while varying scalarizations for a single output can create instability. Together, they form a stable set-level objective that directly rewards diversity. Intuitively, VPO turns policy optimization into a coverage problem over the Pareto front.

Reward estimation As it only specifies a reward calculation, VPO can be combined with any policy-gradient method. In our experiments, we use GRPO [Shao et al., 2024]. For each prompt x , we sample G rollouts, each producing a set $S^{(g)} = \{y_1^{(g)}, \dots, y_m^{(g)}\}$ of m completions, along with K scalarization weights $w^{(1)}, \dots, w^{(K)} \stackrel{\text{iid}}{\sim} \text{Dir}(\mathbf{1})$ used in common across the group. The per-rollout Monte-Carlo reward is

$$\hat{R}(S^{(g)}) = \frac{1}{K} \sum_{k=1}^K \max_{s \in S^{(g)}} w^{(k)\top} r(x, s),$$

which estimates $R(S)$ from the previous section. The GRPO advantage is then calculated and applied uniformly to every token in the rollout g . The K scalarization weights are shared across the G rollouts in the group, so all G sets are evaluated under the same draws of w and are therefore comparable.

4 Experimental Setup

4.1 Evaluations Tasks

We evaluate on four domains chosen to span distinct shapes of multi-objective structure: (i) binary vs. continuous reward components, and (ii) hand-crafted vs. metric-based reward shapes. In all our experiments, we used $m=3$ candidates per multi-answer chain across all domains. Full reward and prompt details are in App. A.

Maze. A synthetic 9×9 navigation task in which the model emits, in text, a sequence of moves from a start corner S to a goal corner E , collecting gold and diamond items and avoiding lava along the way. We construct the mazes so that item types are forced to trade off against each other and against reaching the exit: the geometry guarantees that no single route can satisfy every reward component. The reward $r \in \mathbb{R}^4$ has one binary completion component and three clipped item/safety terms (gold, diamond, lava-avoidance); the GRPO scalar is the uniform mean. The maze is our controlled testbed: unlike the other three domains, the trade-off is engineered rather than naturally occurring, which lets us isolate the counter-intuitive question of whether training VPO and evaluating under the *same* uniform mean GRPO was directly optimized for still beats GRPO. We train Qwen3-4B and evaluate on 100 held-out mazes.

MuSiQue. A 2–4 hop reading-comprehension benchmark [Trivedi et al., 2022] in which the model selects supporting paragraphs from 20 candidates and emits a final answer. The reward $r \in \mathbb{R}^5$ has four binary citation indicators (one per gold hop) plus a continuous answer-F1 term; the GRPO scalarization weights the answer $3 \times$ to reflect its priority. We train Qwen3-1.7B and evaluate on a 300-question hop-stratified split.

EUREQA. A 5-hop chain-reasoning benchmark [Li et al., 2024] where the model back-chains through five relations to identify five masked entities. The reward $r \in \{0, 1\}^5$ is binary per-entity and the GRPO scalar is a uniform mean. Unlike MuSiQue’s loosely coupled hops, the chain is causally ordered, so the per-hop training signal is informative about which step failed. We train Qwen3-8B and evaluate on a held-out hard split. Due to the limited size of the eval dataset, we averaged over 4 evaluation seeds

ToolRL. A function-calling benchmark [Qian et al., 2025] of 3,920 train and 80 test prompts. The reward $r \in \mathbb{R}^4$ has one binary structural-format component and three continuous F1 dimensions (tool-name, arg-key, arg-value), graded from trivially solvable to schema-precise; the GRPO scalar is

a uniform mean. We train Qwen3-1.7B and evaluate on the 80-prompt test split. Due to the limited size of the eval dataset, we averaged over 4 evaluation seeds.

We build on veRL Sheng et al. [2024] with standard outcome-reward GRPO [Shao et al., 2024]. The full training details are documented in App. B.

4.2 Baselines

Our baselines are designed to isolate which ingredient of VPO is responsible for improved test-time search. VPO combines two mechanisms: multi-answer generation within a single autoregressive rollout, and stochastic scalarization over a vector-valued reward. We compare against methods that test whether either ingredient, or existing search-aware RL objectives, are sufficient on their own.

- **GRPO** [Shao et al., 2024] (single-answer, scalar reward). This is the standard RL post-training baseline. Rewards are collapsed into a fixed scalar objective, and the model emits one completion per prompt. This tests whether ordinary scalar RL already produces candidate pools that are useful for test-time search.
- **Multi-RLVR** [Puri et al., 2026] (multi-answer, scalar reward). This baseline trains the model to emit multiple answers in one rollout, but evaluates the set using the same fixed scalar reward. It tests whether multi-answer generation alone is sufficient, or whether a diversity-preserving reward signal is also needed.
- **Random-Weighting GRPO** (single-answer, stochastic scalarization). Here the model still emits one answer per rollout, but the scalarization weights are resampled during training, $w \sim \text{Dir}(\alpha)$. This tests whether randomizing the reward objective alone is enough without set-level optimization.
- **Max-at- k Training** [Bagirov et al., 2025]. This baseline directly optimizes an inference-aware $\text{best@}k/\text{max@}k$ objective. It tests whether explicitly training for best-of- k performance is sufficient without requiring reward-diverse candidate sets.
- **MaxRL**. MaxRL [Tajwar et al., 2026] is another search-aware RL objective that uses additional sampling compute during training to better approximate maximum-likelihood-style learning from successful rollouts. It tests whether stronger scalar search-aware training objectives can recover the gains of VPO.
- **Goal-Conditioned GRPO**. This baseline conditions the policy on a target scalarization w and trains it to maximize $w^\top r(x, y)$. It tests the natural multi-objective RL alternative: whether diversity is better obtained by asking for different trade-offs explicitly, rather than by producing a reward-diverse set within one rollout.

4.3 Evaluation Metrics

Best@ k . Our central metric is the maximum scalarized reward over a pool of k candidates,

$$\text{best@}k(x) = \max_{s \in S_k(x)} w^{\star\top} r(x, s),$$

where w^* is the per-domain GRPO training scalar (Table 7) and $S_k(x)$ is a pool of k completions sampled from the trained policy on prompt x . Multi-answer methods draw $\lceil k/m \rceil$ independent multi-answer chains, each yielding m candidates, and concatenate them in draw order; S_k is the first k entries of this list, so S_k for $k \leq m$ comes from a single chain. Single-answer methods draw k i.i.d. completions.

Reward-space diversity. Alongside $\text{best@}k$, we report the average pairwise L_1 distance between the *reward vectors* of the completions in the pool, which measures spread in the reward space $\mathbb{R}^d$: high diversity means the candidates realize different reward trade-offs. A model whose pool collapses to a single mode has $\text{div} \rightarrow 0$ even when the surface text varies.

5 Results

VPO Improves Gains from Test-Time Search A central goal of VPO is to improve the effectiveness of test-time search by producing more diverse and useful candidate solutions. We evaluate this

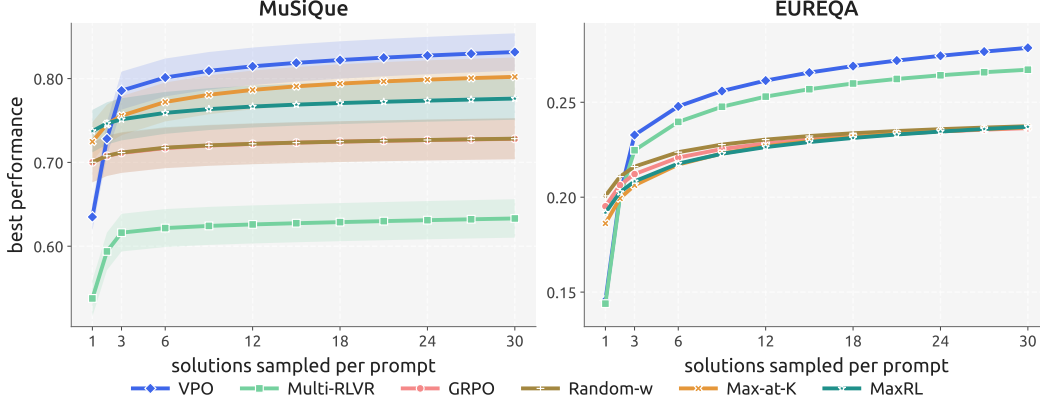


Figure 3: **Test-time scaling on MuSiQue and EUREQA.** Best@ k on the GRPO training scalar as a function of k . Scalar GRPO plateaus quickly, reflecting collapse of the candidate pool to near-duplicates, while VPO continues to extract value from additional samples.

through best@ k , a simple search procedure that chooses the maximum scalarized reward over a pool of k candidates.

Across all four domains, Maze (Table 2), MuSiQue (Table 1), EUREQA (Table 3), and ToolRL (Table 4), VPO consistently improves best@ k relative to scalar baselines. While scalar GRPO quickly saturates as k increases, indicating collapsing candidate diversity, VPO continues improving with additional samples and plateaus at a substantially higher level. Figure 3 shows this effect on MuSiQue and EUREQA (the companion plot for Maze and ToolRL is in App. D).

Table 1: **Best@ k on MuSiQue** (held-out 300-question hop-stratified split). Bold marks the best entry per column, underline the second-best. VPO continues improving as k grows while scalar baselines plateau.

Method	best@3	best@5	best@10	best@30	F1@30	diversity
Qwen3-1.7B (single-answer prompt)	0.310	0.339	0.373	0.423	0.161	0.634
↪ GRPO	0.711	0.716	0.721	0.728	0.447	0.054
↪ Random-w	0.712	0.716	0.721	0.728	0.445	0.061
↪ Max-at-K	0.757	0.768	0.783	0.802	0.573	0.175
↪ MaxRL	<u>0.751</u>	0.757	0.765	0.776	0.552	0.068
Qwen3-1.7B (multi-answer prompt)	0.402	0.439	0.475	0.506	0.379	0.855
↪ Multi-RLVR	0.599	0.616	0.627	0.633	0.498	0.814
↪ VPO (ours)	0.742	0.780	0.809	0.832	0.678	<u>0.587</u>

Table 2: **Best@ k on Maze** (100 held-out mazes). VPO outperforms scalar baselines under the same uniform-mean GRPO scalar even though it was trained on Dirichlet-sampled scalarizations.

Method	best@3	best@5	best@10	best@30	diversity
Qwen3-4B (single-answer prompt)	0.341	0.457	0.596	0.714	0.905
↪ GRPO	0.432	0.432	0.432	0.432	0.003
↪ Random-w	0.414	0.420	0.425	0.432	0.090
↪ Max-at-K	0.526	<u>0.552</u>	<u>0.568</u>	<u>0.577</u>	<u>0.671</u>
↪ MaxRL	0.414	0.426	0.441	0.464	0.206
Qwen3-4B (multi-answer prompt)	0.066	0.103	0.178	0.334	0.176
↪ Multi-RLVR	0.420	0.430	0.435	0.436	0.187
↪ VPO (ours)	<u>0.512</u>	0.564	0.591	0.593	1.006

Is the gain due to multi-answer prompting alone? We want to understand if generating multiple answers within a shared autoregressive context, as explored in [Puri et al., 2026] is enough for

Table 3: **Best@ k on EUREQA** (held-out half of `hard_5`, averaged over 4 evaluation seeds).

Method	best@3	best@5	best@10	best@30	diversity
Qwen3-8B (single-answer prompt)	0.081	0.096	0.117	0.153	0.171
↪ GRPO	0.212	0.219	0.226	0.236	0.119
↪ Random-w	0.216	0.222	0.229	0.237	0.105
↪ Max-at-K	0.206	0.214	0.224	0.237	0.140
↪ MaxRL	0.209	0.216	0.224	0.237	0.117
Qwen3-8B (multi-answer prompt)	0.102	0.113	0.126	0.140	0.171
↪ Multi-RLVR	0.210	<u>0.230</u>	<u>0.249</u>	<u>0.267</u>	0.526
↪ VPO (ours)	<u>0.213</u>	0.236	0.257	0.279	<u>0.512</u>

Table 4: **Best@ k on ToolRL** (80-prompt held-out split, averaged over 4 evaluation seeds).

Method	best@3	best@5	best@10	best@30	diversity
Qwen3-1.7B (single-answer prompt)	0.112	0.114	0.116	0.123	0.026
↪ GRPO	<u>0.921</u>	0.923	0.924	0.925	0.044
↪ Random-w	0.902	0.904	0.906	0.907	0.064
↪ Max-at-K	0.940	0.945	<u>0.949</u>	0.954	0.131
↪ MaxRL	0.779	0.805	0.826	0.846	<u>0.510</u>
Qwen3-1.7B (multi-answer prompt)	0.093	0.104	0.111	0.116	0.198
↪ Multi-RLVR	0.861	0.883	0.898	0.905	0.308
↪ VPO (ours)	0.897	<u>0.934</u>	0.950	<u>0.952</u>	1.297

diversity. It does not. VPO outperformed Multi-RLVR on best@ k across all four domains, and the gap widened with k , mirroring the pattern against scalar GRPO. The mechanism is visible in App. D, Fig. 6, which plots the pairwise L_1 distance between the per-rollout *reward vectors* in each candidate pool—a measure of how much the pool spreads in reward space, not in token space. Throughout training, Multi-RLVR’s reward-space diversity collapses on Maze, MuSiQue, and ToolRL. Interestingly, the domain on which Multi-RLVR performs best relative to the other baselines (EUREQA) is also the only one on which its training-time reward-space diversity tracks VPO’s, supporting the hypothesis that VPO’s training-time diversity is instrumental in improving test-time performance. We conclude that multi-answer prompting supplies the capacity to produce distinct candidates, but under a fixed scalarization, the gradient still pushes every position in the chain toward the same scalar optimum, so the candidates collapse to similar reward vectors. The stochastic scalarization in VPO gives different positions an incentive to specialize.

Is the gain due to more evaluator signal during training or normalization issues? Two natural questions arise. First, VPO may benefit simply from receiving 3x more evaluator signal per rollout. Second, GRPO is known to be sensitive to reward components with very different variances, where a single high-variance dimension can dominate the baseline-subtracted advantage. GDPO [Liu et al., 2026] addresses this by normalizing the advantage per reward dimension before aggregating. If VPO’s gains came from better gradient conditioning across components rather than from set-level diversity, per-dimension normalization should close the gap.

We test both at once on the MuSiQue domain by giving GRPO and GDPO 3x the rollouts ($n = 24$) to equalize evaluator calls with VPO at $n = 8$. Note that this also gives the baselines 3x the LM compute, since their reasoning chains are independent per answer rather than shared, so the comparison is conservative against VPO. Table 5 shows that neither effect explains the gap. At matched $n = 8$, GDPO closely tracks GRPO, indicating the per-component normalization is not a binding constraint. At 3x the compute, both GRPO and GDPO improve modestly but remain below VPO at $n = 8$, and adding random w scalarization at $n = 24$ also fails to close the gap. Extra evaluator signal during

Table 5: MuSiQue. Even when GRPO/GDPO receive $3\times$ the rollouts ($n=24$, also $3\times$ the LM compute), they don’t match VPO at $n=8$.

Method	best@3	$\mathbb{E}_w[\text{best@3}]$
GRPO ($n=8$)	0.741	0.830
GDPO ($n=8$)	0.737	0.832
GRPO ($n=24, 3\times$)	0.763	0.841
GDPO ($n=24, 3\times$)	0.765	0.850
GRPO + rand- w ($n=24$)	0.755	0.839
VPO (ours, $n=8$)	0.779	0.856

training and better normalization help marginally, but scalarized objectives still remove the incentive to maintain a diverse candidate set.

Why not just condition on w ?

To optimize the model to output responses on the Pareto front, VPO uses in-context exploration. Prior work instead trains goal-conditioned policies [Yang et al., 2024, Mahankali et al., 2024] that take a target scalarization w as input and optimize $w^\top r$ directly. (These

approaches create diversity by varying w at inference time). To compare, we train a goal-conditioned GRPO policy and evaluate it under two settings (Table 6): (i) conditioning on the canonical scalarization w^* and (ii) conditioning on random $w \sim \text{Dir}(\mathbf{1})$. Neither matches VPO on the maze domain under either gold scalar expected value under a random weight distributions. Interestingly, the goal conditioned policy had mode collapsed (hence identical best@3 and best@6), and also began to ignore the conditioning. Despite explicit access to w , the model struggles to reliably translate text-encoded preferences into effective behavior.

Table 6: **Goal-conditioned GRPO on Maze** under fixed w^* and random $w \sim \text{Dir}(\mathbf{1})$. Neither matches VPO on either the gold scalar or the Dirichlet-averaged metric.

Method	best@3	best@6	$\mathbb{E}_w[\text{best@3}]$	$\mathbb{E}_w[\text{best@6}]$
G.C. $w = w^*$	0.205	0.205	0.201	0.201
G.C. $w \sim \text{Dir}(\mathbf{1})$	0.205	0.205	0.201	0.201
VPO (ours)	0.512	0.576	0.512	0.584

How does VPO scale to harder problems and more sophisticated search?

Two questions remain after the main results: does VPO’s benefit survive on substantially harder problems, and does it survive under search procedures more sophisticated than best@ k ? We answer both in a single-checkpoint case study comparing VPO to scalar GRPO on LiveCodeBench (LCB) [Jain et al., 2025], a competitive-programming benchmark with a strict temporal held-out cut (App. A.5). Both runs train on the same DeepCoder corpus from the same Qwen2.5-Coder-7B-Instruct checkpoint for one epoch; the only difference is the advantage estimator. On single-shot pass@1 (Fig. 4A), the regime with no downstream search to amortize over, GRPO is better; the scalar baseline correctly wins when only one shot is allowed. The moment the model is given a candidate chain of $m=3$ and is evaluated under best@ k (Fig. 4B), the picture inverts: VPO sits above GRPO at every k and the gap widens with k , mirroring the main benchmark results. Replacing best@ k with a more capable search procedure makes the case sharper still. We plug both checkpoints into OpenEvolve [Sharma, 2025], an evolutionary test-time search loop that iteratively rewrites candidates against test feedback, and run it on the 32 hardest held-out problems (those on which neither method passes any test case at best@30). Over 200 search iterations VPO continues to discover new solutions and cracks problems that neither arm could touch under the standard regime, while GRPO plateaus early (Fig. 4C,D). The case study points the same direction as the main results: diversity matters most when downstream search is non-trivial, and the benefit sharpens both as the problems get harder and as the search procedure gets more capable.

When would VPO not help?

VPO’s central premise is that the reward decomposes into components whose individually optimal responses occupy distinct regions of the simplex, and its empirical gain is largest precisely when this premise holds. On Maze, MuSiQue, EUREQA, and ToolRL the on-policy reward components are not collinear under the trained model (see App. F), the Pareto front is wide, and VPO produces sets that span it. The benefit shrinks as the reward becomes effectively scalar. On a separate experiment using the UltraFeedback [Cui et al., 2023] under ArmoRM-5 reward model [Wang et al., 2024] the five nominally distinct components are near-collinear, and so the simplex collapses to a near-line, and VPO converges below scalar baselines on absolute best@ k while still retaining the largest per- K headroom of any method.

6 Related Work

Inference-time search over LLM outputs

A line of work treats the LLM as a generator inside a search loop. Best-of- N sampling [Beirami et al., 2025], self-consistency [Wang et al., 2022], particle filtering [Puri et al., 2025], and Tree-of-Thoughts [Yao et al., 2023] draw or expand many candidates and select among them. AlphaEvolve [Novikov et al., 2025] and FunSearch [Romera-Paredes et al., 2024] wrap static LLMs in evolutionary outer loops. The effectiveness of these approaches depends on generating candidate sets with sufficient diversity and quality for selection to improve outcomes.

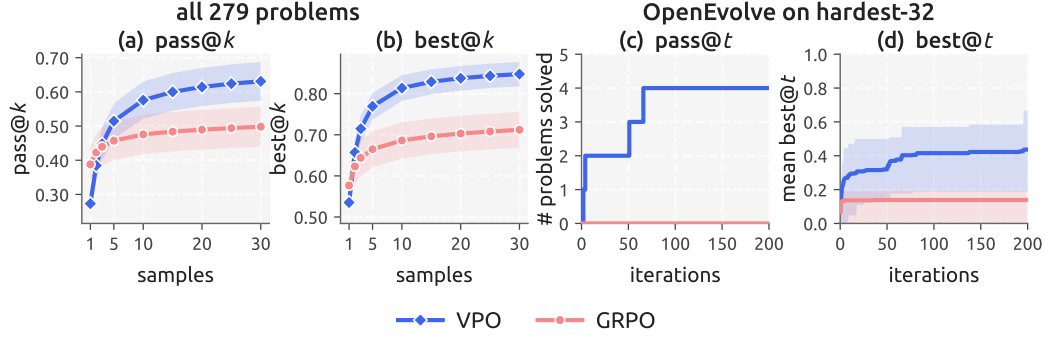


Figure 4: **LiveCodeBench case study: VPO vs. scalar GRPO.** (A) Pass@ k on the full 279-problem held-out split: at $k=1$ GRPO is better, but VPO catches up and overtakes as k grows. (B) Best@ k on the same split: VPO sits above GRPO at every k and the gap widens with k , mirroring the main benchmark results. (C, D) Pass@ k and best@ k over OpenEvolve search iterations on the 32 hardest held-out problems (those on which both methods score 0 at best@30 in the standard regime). VPO continues to discover new solutions over 200 iterations and cracks problems neither arm could touch under best@ k , while GRPO plateaus early.

VPO instead trains the policy to directly produce candidate sets that better support inference-time selection. Another line of work integrate training into the search procedure [Yuksekgonul et al., 2026, Wang et al., 2025]. These methods deliberately sacrifice generalization, as the policy is overfit to one instance for the duration of the search which is out of the scope of our setting.

Training for best@ k A growing line of work modifies the training objective to anticipate inference-time selection. BOND [Sessa et al., 2025] and BoNBoN [Gui et al., 2024] distill the Best-of-N distribution into a single policy. InfAlign [Balashankar et al., 2024] derives reward transformations that make standard RLHF inference-aware for procedures like BoN. Closer to our setting, BoN-aware fine-tuning [Chow et al., 2025], PKPO [Walder and Karkhanis, 2025], Pass@ k training [Chen et al., 2025], and max@ k optimization [Bagirov et al., 2025] derive objectives that directly optimize max@ k (or pass@ k for binary rewards). These methods treat the k samples as independent draws from the policy, whereas VPO emits the m candidates as a single autoregressive chain. We additionally compare to Multi-RLVR [Puri et al., 2026], which jointly generates a set of k outputs but optimizes a fixed scalar objective, whereas VPO optimizes across sampled reward weightings. Our experiments show that VPO outperforms these methods, including max@ k training [Bagirov et al., 2025].

Multi-objective RL and reward randomization Vector rewards and Pareto-optimal policy sets are classical tools in learning [Roijers et al., 2013, Hayes et al., 2022]. Goal-conditioned methods recover the Pareto front by training a single network conditioned on a sampled weight vector [Yang et al., 2019, Kanazawa and Gupta, 2023]. Reward randomization has also been used to escape local optima [Mahankali et al., 2024]. Tang et al. [2021] sample reward perturbations during training to discover diverse multi-agent strategies. VPO builds on these ideas and proposes an RL algorithm to optimize a multi-objective reward that fits and leverages the capabilities of LLMs.

Diversity collapse from RL post-training RL post-training sharpens the output distribution and erodes pass@ k at large k [Kirk et al., 2024, Yue et al., 2025, Karouzos et al., 2026]. GX-Chen et al. [2025] argue this is structural. VPO instead changes the objective, so coverage of the reward simplex is the equilibrium rather than something a regularizer fights for.

7 Discussion and Conclusion

We argued that when language models are deployed inside pipelines with test-time search, the responsibilities of exploration and exploitation should be separated: training should produce a diverse pool of competent candidates, and the search procedure at test time should handle exploitation. VPO instantiates this by sampling scalarizations uniformly over the simplex and training the policy to

emit sets that span the Pareto front of the underlying reward components. The change is a drop-in replacement for the GRPO advantage estimator. Across MuSiQue, EUREQA, Maze, and ToolRL, VPO improves best@ k over scalar baselines, with the gap widening as the test-time budget grows.

Why does VPO work? We see two complementary explanations. First, policies that are optimized to maximize the set-level reward $R(S)$ may produce reward-diverse sets that cover the Pareto front (or its budget-constrained subset). The second possible explanation is about training dynamics. A candidate that scores poorly under w^* but well under some other w still receives a positive gradient on the rollouts where w is sampled, while a fixed- w^* run would push it away. VPO, therefore, could keep a broader set of reasoning strategies alive long enough to be refined, including strategies a w^* -trained policy would never visit. Although hard to measure, this could be a part of why VPO outperforms a w^* -trained policy even when both are evaluated under w^* .

Limitations VPO comes with three limitations. First, precisely equalizing training compute across methods is non-trivial, since each method produces outputs of different lengths and VPO generates m completions per rollout rather than one (though this is partly amortized because the reasoning prefix is shared across the m solutions). To address this, we show in Section 5 that VPO continues to outperform GRPO on MuSiQue even when GRPO is given 3x the compute, indicating that the gains are not merely a function of compute mismatch. Second, VPO benefits from a vector-valued reward; when reward is scalar only, it reduces to more standard RL. Finally, it sacrifices pass@1 for pass@ k by training the policy to explore rather than to exploit. VPO is for the regime where test-time search is part of the system.

Acknowledgments and Disclosure of Funding

The authors express gratitude to Jyo Pari, Nitish Dashora, Andre Ye, Itamar Pres, Navodita Sharma, John Marangola, Nolan Fey, Oliver Sieberling, Linlu Qiu, Luca Grillotti, Benjamin Eysenbach, Yoon Kim, Jacob Andreas, and members of the Improbable AI lab at MIT for discussions that helped shape this work. This work was supported by National Science Foundation graduate research fellowships to RB, IP, and AK, the MIT-IBM Computing Research Lab, the MIT-Google Program for Computing Innovation and Hyundai Motor Company. This research was sponsored by the Army Research Office and was accomplished under Grant Numbers W911NF2110328 and W911NF-23-1-0277. The views and conclusions contained in this document are those of the authors and should not be interpreted as representing the official policies, either expressed or implied, of the Army Research Office or the U.S. Government. The U.S. Government is authorized to reproduce and distribute reprints for Government purposes notwithstanding any copyright notation herein.

References

- Farid Bagirov, Mikhail Arkhipov, Ksenia Sycheva, Evgeniy Glukhov, and Egor Bogomolov. The best of n worlds: Aligning reinforcement learning with best-of- n sampling via max@ k optimisation. *arXiv preprint arXiv:2510.23393*, 2025.
- Ananth Balashankar, Ziteng Sun, Jonathan Berant, Jacob Eisenstein, Michael Collins, Adrian Hutter, Jong Lee, Chirag Nagpal, Flavien Prost, Aradhana Sinha, et al. Infalign: Inference-aware language model alignment. *arXiv preprint arXiv:2412.19792*, 2024.
- Ahmad Beirami, Alekh Agarwal, Jonathan Berant, Alexander Nicholas D’Amour, Jacob Eisenstein, Chirag Nagpal, and Ananda Theertha Suresh. Theoretical guarantees on the best-of- n alignment policy. In *Forty-second International Conference on Machine Learning*, 2025. URL <https://openreview.net/forum?id=u3U8qzFV7w>.
- Marc Bellemare, Sriram Srinivasan, Georg Ostrovski, Tom Schaul, David Saxton, and Remi Munos. Unifying count-based exploration and intrinsic motivation. *Advances in neural information processing systems*, 29, 2016.
- Bradley Brown, Jordan Juravsky, Ryan Ehrlich, Ronald Clark, Quoc V Le, Christopher Ré, and Azalia Mirhoseini. Large language monkeys: Scaling inference compute with repeated sampling. *arXiv preprint arXiv:2407.21787*, 2024.

- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- Zhipeng Chen, Xiaobo Qin, Youbin Wu, Yue Ling, Qinghao Ye, Wayne Xin Zhao, and Guang Shi. Pass@ k training for adaptively balancing exploration and exploitation of large reasoning models. *arXiv preprint arXiv:2508.10751*, 2025.
- Yinlam Chow, Guy Tennenholtz, Izzeddin Gur, Vincent Zhuang, Bo Dai, Sridhar Thiagarajan, Craig Boutilier, Rishabh Agarwal, Aviral Kumar, and Aleksandra Faust. Inference-Aware Fine-Tuning for Best-of-N Sampling in Large Language Models, November 2025. URL <http://arxiv.org/abs/2412.15287>. arXiv:2412.15287 [cs].
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- Ganqu Cui, Lifan Yuan, Ning Ding, Guanming Yao, Bingxiang He, Wei Zhu, Yuan Ni, Guotong Xie, Ruobing Xie, Yankai Lin, et al. Ultrafeedback: Boosting language models with scaled ai feedback. *arXiv preprint arXiv:2310.01377*, 2023.
- Lin Gui, Cristina Gârbacea, and Victor Veitch. Bonbon alignment for large language models and the sweetness of best-of-n sampling. *Advances in Neural Information Processing Systems*, 37: 2851–2885, 2024.
- Anthony GX-Chen, Jatin Prakash, Jeff Guo, Rob Fergus, and Rajesh Ranganath. Kl-regularized reinforcement learning is designed to mode collapse. *arXiv preprint arXiv:2510.20817*, 2025.
- Conor F. Hayes, Roxana Rădulescu, Eugenio Bargiacchi, Johan Källström, Matthew Macfarlane, Mathieu Reymond, Timothy Verstraeten, Luisa M. Zintgraf, Richard Dazeley, Fredrik Heintz, Enda Howley, Athirai A. Irissappane, Patrick Mannion, Ann Nowé, Gabriel Ramos, Marcello Restelli, Peter Vamplew, and Diederik M. Roijers. A practical guide to multi-objective reinforcement learning and planning. *Autonomous Agents and Multi-Agent Systems*, 36(1), April 2022. ISSN 1573-7454. doi: 10.1007/s10458-022-09552-y. URL <http://dx.doi.org/10.1007/s10458-022-09552-y>.
- Zhang-Wei Hong, Idan Shenfeld, Johnson Wang, Yung-Sung Chuang, Aldo Pareja, James R Glass, Akash Srivastava, and Pulkit Agrawal. Curiosity-driven red-teaming for large language models. In *International Conference on Learning Representations*, volume 2024, pages 19935–19961, 2024.
- Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=chfJJYC3iL>.
- Takuya Kanazawa and Chetan Gupta. Latent-conditioned policy gradient for multi-objective deep reinforcement learning. In *International Conference on Artificial Neural Networks*, pages 63–76. Springer, 2023.
- Constantinos Karouzos, Xingwei Tan, and Nikolaos Aletras. Where does output diversity collapse in post-training? *arXiv preprint arXiv:2604.16027*, 2026.
- Robert Kirk, Ishita Mediratta, Christoforos Nalmpantis, Jelena Luketina, Eric Hambro, Edward Grefenstette, and Roberta Raileanu. Understanding the effects of RLHF on LLM generalisation and diversity. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=PX3FAVHJT>.
- William La Cava, Thomas Helmuth, Lee Spector, and Jason H. Moore. A Probabilistic and Multi-Objective Analysis of Lexicase Selection and ϵ -Lexicase Selection. *Evolutionary Computation*, 27(3):377–402, 2019. ISSN 1530-9304. doi: 10.1162/evco_a_00224.
- Pawel Ladosz, Lilian Weng, Minwoo Kim, and Hyondong Oh. Exploration in deep reinforcement learning: A survey. *Information Fusion*, 85:1–22, 2022.

- Robert Tjarko Lange, Yuki Imajuku, and Edoardo Cetin. Shinkaevolve: Towards open-ended and sample-efficient program evolution. *arXiv preprint arXiv:2509.19349*, 2025.
- Bangzheng Li, Ben Zhou, Fei Wang, Xingyu Fu, Dan Roth, and Muhao Chen. Deceptive semantic shortcuts on reasoning chains: How far can models go without hallucination? In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 7675–7688, 2024.
- Shih-Yang Liu, Xin Dong, Ximing Lu, Shizhe Diao, Peter Belcak, Mingjie Liu, Min-Hung Chen, Hongxu Yin, Yu-Chiang Frank Wang, Kwang-Ting Cheng, Yejin Choi, Jan Kautz, and Pavlo Molchanov. GDPO: Group reward-Decoupled Normalization Policy Optimization for Multi-reward RL Optimization, January 2026. URL <http://arxiv.org/abs/2601.05242>. arXiv:2601.05242 [cs].
- Srinath Mahankali, Zhang-Wei Hong, Ayush Sekhari, Alexander Rakhlin, and Pulkit Agrawal. Random latent exploration for deep reinforcement learning. *arXiv preprint arXiv:2407.13755*, 2024.
- Andrew Ni, Li Ding, and Lee Spector. Dalex: Lexicase-like selection via diverse aggregation. In *European Conference on Genetic Programming (Part of EvoStar)*, pages 90–107. Springer, 2024.
- Alexander Novikov, Ng  n V  , Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt W  gner, Sergey Shirobokov, Borislav Kozlovskii, Francisco JR Ruiz, Abbas Mehrabian, et al. Alphaevolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*, 2025.
- Deepak Pathak, Pulkit Agrawal, Alexei A. Efros, and Trevor Darrell. Curiosity-driven Exploration by Self-supervised Prediction, May 2017. URL <http://arxiv.org/abs/1705.05363>. arXiv:1705.05363 [cs, stat].
- Isha Puri, Shivchander Sudalairaj, Guangxuan Xu, Kai Xu, and Akash Srivastava. Rollout roulette: A probabilistic inference approach to inference-time scaling of llms using particle-based monte carlo methods, 2025. URL <https://arxiv.org/abs/2502.01618>.
- Isha Puri, Mehul Damani, Idan Shenfeld, Marzyeh Ghassemi, Jacob Andreas, and Yoon Kim. Reaching Beyond the Mode: RL for Distributional Reasoning in Language Models, March 2026. URL <http://arxiv.org/abs/2603.24844>. arXiv:2603.24844 [cs].
- Cheng Qian, Emre Can Acikgoz, Qi He, Hongru Wang, Xiushi Chen, Dilek Hakkani-T  r, Gokhan Tur, and Heng Ji. Toolrl: Reward is all tool learning needs. *arXiv preprint arXiv:2504.13958*, 2025.
- Yuxiao Qu, Amrith Setlur, Virginia Smith, Ruslan Salakhutdinov, and Aviral Kumar. Pope: Learning to reason on hard problems via privileged on-policy exploration. *arXiv preprint arXiv:2601.18779*, 2026.
- Diederik M Roijers, Peter Vamplew, Shimon Whiteson, and Richard Dazeley. A survey of multi-objective sequential decision-making. *Journal of Artificial Intelligence Research*, 48:67–113, 2013.
- Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M Pawan Kumar, Emilien Dupont, Francisco JR Ruiz, Jordan S Ellenberg, Pengming Wang, Omar Fawzi, et al. Mathematical discoveries from program search with large language models. *Nature*, 625(7995):468–475, 2024.
- Pier Giuseppe Sessa, Robert Dadashi-Tazehozzi, L  onard Hussenot, Johan Ferret, Nino Vieillard, Alexandre Ram  , Bobak Shahriari, Sarah Perrin, Abram L Friesen, Geoffrey Cideron, et al. Bond: Aligning llms with best-of-n distillation. In *The Thirteenth International Conference on Learning Representations*, 2025.
- Amrith Setlur, Matthew YR Yang, Charlie Snell, Jeremy Greer, Ian Wu, Virginia Smith, Max Simchowitz, and Aviral Kumar. e3: Learning to explore enables extrapolation of test-time compute for llms. *arXiv preprint arXiv:2506.09026*, 2025.

- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- Asankhaya Sharma. Openevolve: an open-source evolutionary coding agent, 2025. URL <https://github.com/algorithmicsuperintelligence/openevolve>.
- Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. Hybridflow: A flexible and efficient rlhf framework. *arXiv preprint arXiv: 2409.19256*, 2024.
- Lee Spector. Assessment of Problem Modality by Differential Performance of Lexicase Selection in Genetic Programming: A Preliminary Report. 2012.
- Lee Spector, Li Ding, and Ryan Boldi. Particularity. In *Genetic Programming Theory and Practice XX*, pages 159–176. Springer, 2024.
- Richard S Sutton and Andrew G Barto. *Reinforcement learning: An introduction*, volume 1. MIT press Cambridge, 1998.
- Fahim Tajwar, Guanning Zeng, Yueer Zhou, Yuda Song, Daman Arora, Yiding Jiang, Jeff Schneider, Ruslan Salakhutdinov, Haiwen Feng, and Andrea Zanette. Maximum likelihood reinforcement learning. *arXiv preprint arXiv:2602.02710*, 2026.
- Zhenggang Tang, Chao Yu, Boyuan Chen, Huazhe Xu, Xiaolong Wang, Fei Fang, Simon Du, Yu Wang, and Yi Wu. Discovering diverse multi-agent strategic behavior via reward randomization. *arXiv preprint arXiv:2103.04564*, 2021.
- Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. Musique: Multihop questions via single-hop question composition. *Transactions of the Association for Computational Linguistics*, 10:539–554, 2022.
- Christian Walder and Deep Karkhanis. Pass@ k policy optimization: Solving harder reinforcement learning problems. *arXiv preprint arXiv:2505.15201*, 2025.
- Haoxiang Wang, Wei Xiong, Tengyang Xie, Han Zhao, and Tong Zhang. Interpretable preferences via multi-objective reward modeling and mixture-of-experts. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 10582–10592, 2024.
- Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*, 2022.
- Yiping Wang, Shao-Rong Su, Zhiyuan Zeng, Eva Xu, Liliang Ren, Xinyu Yang, Zeyi Huang, Xuehai He, Luyao Ma, Baolin Peng, Hao Cheng, Pengcheng He, Weizhu Chen, Shuohang Wang, Simon Shaolei Du, and Yelong Shen. ThetaEvolve: Test-time Learning on Open Problems, November 2025. URL <http://arxiv.org/abs/2511.23473>. arXiv:2511.23473 [cs].
- Fang Wu and Yejin Choi. The invisible leash: Why rlvr may not escape its origin. In *2nd AI for Math Workshop@ ICML 2025*, 2025.
- Rui Yang, Xiaoman Pan, Feng Luo, Shuang Qiu, Han Zhong, Dong Yu, and Jianshu Chen. Rewards-in-context: Multi-objective alignment of foundation models with dynamic preference adjustment. *arXiv preprint arXiv:2402.10207*, 2024.
- Runzhe Yang, Xingyuan Sun, and Karthik Narasimhan. A generalized algorithm for multi-objective reinforcement learning and policy adaptation. *Advances in neural information processing systems*, 32, 2019.
- Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. *Advances in neural information processing systems*, 36:11809–11822, 2023.

Yang Yue, Zhiqi Chen, Rui Lu, Andrew Zhao, Zhaokai Wang, Yang Yue, Shiji Song, and Gao Huang. Does reinforcement learning really incentivize reasoning capacity in llms beyond the base model?, 2025. URL <https://arxiv.org/abs/2504.13837>.

Mert Yuksekgonul, Daniel Kocaja, Xinhao Li, Federico Bianchi, Jed McCaleb, Xiaolong Wang, Jan Kautz, Yejin Choi, James Zou, Carlos Guestrin, et al. Learning to discover at test time. *arXiv preprint arXiv:2601.16175*, 2026.

A Environment Details

A.1 Maze

Generation. Each maze is a 9×9 grid built in two stages. We first carve a spanning tree using Prim’s algorithm: starting from an all-walls grid and a uniformly random seed cell, we repeatedly pop a uniform random frontier cell and carve it to empty if it has exactly one empty neighbor (the standard wall-carving rule). We then inject $n_{\text{cycles}} \sim \text{Unif}\{18, \dots, 28\}$ additional openings by converting wall cells with ≥ 2 empty neighbors to empty; this is essential, since it turns the spanning tree into a multi-route graph so VPO’s candidate pool has distinct paths to find. Endpoints are placed as one of $\{(0, 0) \rightarrow (8, 8), (0, 8) \rightarrow (8, 0)\}$ with equal probability; the remaining two corners are randomly designated the *gold corner* and *diamond corner*; the center cell $(4, 4)$ is a *bonus tile* that is rendered in the grid (as B) and advertised in the prompt as a score multiplier, but is in fact a distractor: visiting it has no effect on the reward. We then compute via-BFS detour lengths $\text{via_gold} = d(S, \text{gold}) + d(\text{gold}, E)$, $\text{via_diam} = d(S, \text{diam}) + d(\text{diam}, E)$, and $\text{via_both} = d(S, \text{gold}) + d(\text{gold}, \text{diam}) + d(\text{diam}, E)$, and define the step budget $\text{budget} = \max(\text{via_gold}, \text{via_diam}) + 7$. We *reject* the maze unless $\text{via_both} > \text{budget}$ – this is the design lever that guarantees the budget fits one corner detour but not both, so no single route can hit both the gold and diamond corners and still reach E . We then place $n_{\text{gold}} \sim \text{Unif}\{3, \dots, 5\}$ gold cells in the Manhattan-radius-2 ball around the gold corner, the same for diamonds, and $n_{\text{lava}} \sim \text{Unif}\{3, \dots, 5\}$ lava cells in the strict interior $\{(r, c) : 2 \leq r, c \leq 6\}$. A final BFS check verifies that a lava-avoiding path $S \rightarrow E$ exists within budget; otherwise the maze is rejected. The first 1000 mazes that survive (seeds from 42) form the train split; the next 100 (seeds from 4242) form the test split.

Reward. The model’s text output is parsed into a move list and simulated: walls and borders block, the trajectory ends as soon as E is stepped on, lava is walkable but counted, and items only count if collected before reaching E . If E is never reached, the reward vector is $(0, 0, 0, 0)$ on all four dimensions. Otherwise, define the per-item collection rates

$$g = \frac{|\text{distinct gold visited}|}{n_{\text{gold}}}, \quad d = \frac{|\text{distinct diamond visited}|}{n_{\text{diam}}}, \quad \ell = 1 - \frac{|\text{distinct lava stepped}|}{n_{\text{lava}}}.$$

The 4-D reward is

$$r(x, y) = (1, g, d, \ell) \in [0, 1]^4,$$

i.e., binary completion together with the three linear item-collection and lava-avoidance fractions. The gold scalar used by GRPO, MaxRL, GoalCond(w^*) training, and by best@ k evaluation is the uniform mean

$$\bar{r} = \frac{1}{4}(r_1 + r_2 + r_3 + r_4).$$

A.2 MuSiQue

MuSiQue [Trivedi et al., 2022] is a multi-hop QA benchmark in which each 2–4 hop question decomposes into a chain of sub-questions whose answers must be composed; we use the MuSiQue-Ans split (19,938 train / 2,417 test). Each prompt embeds the question alongside 20 paragraphs, 2–4 of which carry gold supporting evidence and the rest sampled as distractors from MuSiQue’s own paragraph pool (no external retrieval). The model emits a `<support>` block (paragraph indices, capped at four distinct entries) and an `<answer>` block. The reward vector $r(x, s) \in \mathbb{R}^5$ contains four binary citation indicators, one per gold hop ($\text{hop}_1, \dots, \text{hop}_4$), each $\{0, 1\}$ for whether `<support>` cites that paragraph, plus a continuous `answer_f1` against gold (best over aliases, with MuSiQue-style normalization: lowercase, drop articles, drop punctuation). The GRPO scalar is $(\sum_i \text{hop}_i + 3 \text{ answer_f1})/7$, weighting the answer $3\times$ to reflect that a correct answer is worth more than any

single citation. The two reward components are causally linked but not redundant: a model can answer correctly while citing distractors, or cite correctly and fail to compose, so the domain probes pipeline coupling between evidence and answer. The asymmetric scalar weighting gives GRPO an explicit single-objective bias toward the answer, while vector-reward methods see all five dimensions at parity. We train Qwen3-1.7B from a shared base for one epoch across all methods and evaluate on a hop-stratified split of 300 held-out questions.

A.3 EUREQA

EUREQA [Li et al., 2024] is a 5-hop chain-reasoning benchmark released with two difficulty levels of the same underlying questions: `questions_normal_5` (1,109 examples, baseline phrasing) and `questions_hard_5` (682 examples, harder rephrasings). We use a 50/50 random split of `hard_5` (seed 0) as the held-out test set, and train on the remaining half of `hard_5` together with all of the easier `normal_5` rephrasings. The test set is therefore in-distribution with the harder half of training data, while the `normal_5` rephrasings provide a small additional easy-data exposure. Each prompt is a narrative containing six masked entities A, B, C, D, E, F connected by a 5-relation chain, with the anchor entity F shown verbatim; the model must back-chain through the relations to identify A through E by their canonical Wikipedia names (e.g., `Robert_Rodriguez`), emitting five numbered tags. The reward vector $r(x, y) \in \{0, 1\}^5$ is binary, with one exact-match indicator per entity after Wikipedia-style normalization. The GRPO scalar is a uniform mean of the five dimensions. Unlike MuSiQue’s loosely coupled hops, the EUREQA chain is causally chained: identifying E requires composing a relation step from F , D requires E , and so on. Scalar reward provides no signal about which hop in the chain failed, while vector-reward methods see per-hop correctness, so this domain probes whether per-dim training signal yields more robust chain-following on the harder rephrasings. We train Qwen3-8B and report `best@k` averaged over 4 evaluation seeds on the held-out `hard_5` test split.

A.4 ToolRL

ToolRL [Qian et al., 2025] is a function-calling benchmark assembled from ToolACE, Hammer, and xLAM, comprising 3,920 train and 80 test prompts. The reward vector $r(x, y) \in \mathbb{R}^4$ contains one binary and three continuous F1 dimensions: `format` $\in \{0, 1\}$ (structural well-formedness; all four format checks collapse to a single binary pass), `tool_name` $\in [0, 1]$ (multiset F1 between predicted and gold tool-call names), `arg_key` $\in [0, 1]$ (mean set-F1 over parameter keys across aligned tool calls, with greedy alignment by name then by key overlap), and `arg_value` $\in [0, 1]$ (token-level F1 between predicted and gold values on aligned keys). The GRPO scalar is a uniform mean of the four dimensions. The dimensions are graded by difficulty: `format` is trivially solvable, `tool_name` requires retrieval, `arg_key` requires schema knowledge, and `arg_value` requires precise content generation. We train Qwen3-1.7B and report `best@k` averaged over 4 evaluation seeds on the 80-prompt test split.

A.5 LiveCodeBench (case study)

LiveCodeBench (LCB) [Jain et al., 2025] is a competitive-programming benchmark with a strict temporal held-out cut: each problem has a contest date and the held-out slice (Aug 2024 – Feb 2025) postdates every training-time problem, ruling out contamination by construction. We use LCB as a two-arm scaling case study (VPO and scalar GRPO only) rather than as one of the four main benchmark domains. Training data is the DeepCoder corpus, 24,269 problems concatenated from three sources: 16,238 from PrimeIntellect SYNTHETIC-1 (stdin, easiest), 7,432 from TACO (Topics in Algorithmic Code Generation; verified competitive-programming slice, middle difficulty), and 599 from LCB-v5 train (May – Jul 2024, hardest). Each row carries a structured array of test cases with `inputs/outputs` (and an `fn_name` field for functional-I/O problems); per-problem test counts are variable, capped at 32 by the preprocessor. The held-out evaluation split is LCB-v5 Aug 2024 – Feb 2025, 279 problems, strictly later than every training-time LCB problem. The reward vector is a per-test-case binary pass indicator, $r(x, y) \in \{0, 1\}^d$, where d is the problem-specific test count; VPO samples a Dirichlet of dimension d per prompt with no zero-padding for advantage computation. The gold scalar is the uniform mean over the actually-present dimensions, which equals the problem’s pass rate. Training uses Qwen2.5-Coder-7B-Instruct, FSDP across $8 \times \text{H100}$ 80 GB, GRPO inner loop with PPO-clip $\varepsilon=0.2$, AdamW $\text{lr}=10^{-6}$ (constant, no warmup), weight decay 0.01,

gradient clip 1.0, KL coefficient 10^{-3} , no entropy bonus; train batch 64, mini-batch 32, $n=8$ rollouts per prompt, generation temperature 0.8, max context 4096 + 4096. One epoch is ≈ 379 steps; we evaluate at `global_step_378` for both methods. The configuration is identical across VPO and GRPO except for the advantage estimator. For `pass@k` and `best@k` we sample 30 candidates per problem at temperature 0.8 over the full 279-problem held-out split. For OpenEvolve we restrict to the 32 hardest problems (those on which both VPO and GRPO score 0 at `best@30` in the standard regime) and run 200 iterations with $m=3$ candidates per iteration (≈ 600 candidates per problem); the test feedback from each iteration is exposed to the model as the search-loop input. The restricted subset isolates the regime where the standard `best@k` pool is exhausted, so any further progress must come from the search procedure itself rather than from drawing more samples.

B Training Setup

All methods share a common GRPO [Shao et al., 2024] backbone implemented on top of veRL [Sheng et al., 2024]; only the advantage estimator changes per method. The recipe below applies to GRPO and to all VPO variants unless stated otherwise.

Advantage estimation. For each prompt we sample a group of $n=8$ rollouts, compute a per-rollout scalar score (per-domain formula in Table 7), and form the advantage as the within-group z -score $\hat{A}_i = (\text{score}_i - \mu_g) / (\sigma_g + \epsilon)$ with $\epsilon=10^{-6}$ and population standard deviation. The advantage is broadcast across all response tokens via the response mask. There is no value/critic network and no GAE.

Objective. Standard PPO-clip with $\epsilon=0.2$ (symmetric, dual-clip $c=3.0$), `ppo_epochs=1`, token-mean loss aggregation, and no entropy bonus. KL is applied as a loss-side regularizer only (no in-reward penalty): low-variance k_3 estimator against a frozen reference equal to the SFT initialization, with coefficient $\beta_{\text{KL}} = 10^{-3}$.

Optimizer. AdamW with learning rate 10^{-6} , $(\beta_1, \beta_2)=(0.9, 0.999)$, weight decay 0.01, gradient clipping at 1.0, no warmup, constant learning-rate schedule.

Engine and rollout sampling. FSDP1 with bf16 mixed precision (parameter dtype fp32). The reference policy is a frozen copy of the actor’s initialization with parameters CPU-offloaded between forward passes. Training rollouts use vLLM at temperature 1.0, `top-p` 1.0, `top-k` -1. In-training validation uses greedy decoding (`do_sample=False`) on all four domains as a low-noise progress signal.

Final-evaluation sampling. The numbers reported in the main paper come from a separate post-training evaluation pass with stochastic decoding, so that `best@k` has a non-degenerate candidate pool to draw from. We use `top_k = -1` everywhere and the following per-domain settings, applied uniformly across every method evaluated on that domain: Maze and MuSiQue at temperature 0.7, `top-p` 1.0; EUREQA and ToolRL at temperature 0.7, `top-p` 0.95.

Per-domain batch sizes. Batch sizes vary with model scale. For Qwen $\leq 4\text{B}$ (Maze, MuSiQue, ToolRL): train batch 128, mini-batch 64, micro-batch 8, $n=8$ rollouts per prompt. For Qwen 7B/8B (EUREQA): train batch 64, mini-batch 32, micro-batch 2, $n=8$. We honor veRL’s divisibility constraints (train $\times n$ divisible by GPU count; train divisible by mini-batch; mini-batch divisible by micro-batch $\times$ GPU count), so each PPO update consumes the full train-batch generation, with train/mini gradient steps per generation.

Per-domain GRPO scalar. Table 7 lists the per-domain scalar reward formulas $w^{\star\top} r(x, y)$ used as the GRPO score. Empty or unparseable responses receive score = 0 in every domain.

C Compute Resources

All experiments ran on NVIDIA H100 GPUs (80 GB SXM). Each training run used a single node with 4 $\times$ H100, with a typical wall-clock duration of approximately 12 hours per run (~ 48 GPU-hours per

Table 7: Per-domain GRPO scalar reward formulas. d is the dimensionality of the underlying vector reward r .

Domain	d	GRPO scalar formula
Maze	4	$\text{mean}(\text{completion}, \text{gold}, \text{diamond}, \text{avoid_lava})$
MuSiQue	5	$(\text{hop}_1 + \text{hop}_2 + \text{hop}_3 + \text{hop}_4 + 3 \text{ answer_f1})/7$
EUREQA	5	$\text{mean}(\text{entity}_A, \text{entity}_B, \text{entity}_C, \text{entity}_D, \text{entity}_E)$
ToolRL	4	$\text{mean}(\text{format}, \text{tool_name}, \text{arg_key}, \text{arg_value})$

run). The reported best@ k numbers come from training one run per (method, domain) cell across the four domains (Maze, MuSiQue, EUREQA, ToolRL) and the five core methods (GRPO, Random- w , Max-at- K , Multi-RLVR, VPO), plus the goal-conditioned baseline on Maze and the $3\times$ -rollout ablation on MuSiQue, totaling roughly 20–25 reported training runs ($\sim 1,000$ GPU-hours). Including preliminary, failed, and hyperparameter-search runs, the full project consumed approximately $2\times$ the reported budget ($\sim 2,000$ GPU-hours total). Inference-time evaluation (best@ k pool generation and the $\bar{\rho}$ collinearity diagnostics in App. F) was orders of magnitude cheaper and is not separately accounted for.

D Additional Figures

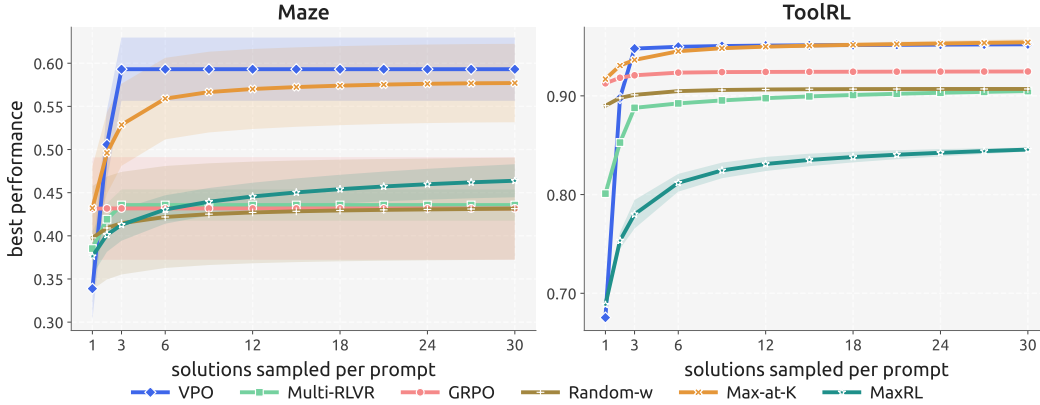


Figure 5: **Test-time scaling on Maze and ToolRL.** Best@ k on the GRPO training scalar as a function of k , pooled across c multi-answer chains per prompt. VPO matches or exceeds scalar baselines at every k on Maze; on ToolRL all methods saturate near the reward ceiling and converge. Companion to Fig. 3.

E Domain Prompts

For each domain, we list the prompt template as it appears in the training corpus (single-solution form) followed by the rewriting applied for multi-solution methods (Multi-RLVR, VPO). We use $m=3$ candidates per multi-answer chain on all domains. Curly-braced names ($\{\text{question}\}$, $\{\text{m}\}$, etc.) are placeholders filled in per example. The goal-conditioning suffix (§E.5) is appended only for the goal-conditioned baseline.

E.1 MuSiQue

MuSiQue: single-solution prompt

Read the paragraphs below and answer the multi-hop question. Identify which paragraphs support your answer.



Figure 6: **Reward-space diversity over training.** Pairwise L_1 distance between the per-rollout reward vectors $r(x, y) \in \mathbb{R}^d$ in each candidate pool, averaged across prompts, plotted over training steps. This measures the spread of the pool in reward space (not in token space): a large value means the rollouts realize different reward trade-offs. VPO sustains substantially higher reward-space diversity than Multi-RLVR throughout training; the only domain on which Multi-RLVR’s diversity tracks VPO’s is EUREQA, which is also the only domain on which Multi-RLVR comes close to VPO on best@ k .

Paragraphs:
{paragraphs_text}

Question: {question}

First reason about which paragraphs are relevant, then output your supporting paragraph indices and answer.

<support>comma-separated paragraph indices (e.g., 3, 7, 12)</support>

<answer>your answer</answer>

{paragraphs_text} is "[{idx}] (Title: {title}) {paragraph_text}" joined over 20 paragraphs (2–4 gold, the rest distractors).

Multi-solution rewrite. The block from “First reason...” through “<answer>your answer</answer>” is replaced with:

MuSiQue: multi-solution rewrite

Give {m} different answers in <response_i>...</response_i> tags, each with <support>indices</support> and <answer>answer</answer>.

The header, paragraphs, and question are unchanged.

E.2 EUREQA

EUREQA: single-solution prompt

You will be given a multi-hop reasoning question over a small narrative. The narrative contains masked entities (Person A, Country B, etc.) connected by relations. The leaf entity is revealed in the final sentence; you must back-chain to identify the rest.

Narrative:
{narrative}

The masked entities you must resolve (in order of introduction in the narrative):
{mask_list}

For each masked entity above, output its underlying canonical name. Use underscored Wikipedia-style names exactly as they appear in the source (e.g. `Robert\_Rodriguez`, `32nd\_European\_Film\_Awards`, `From\_Dusk\_till\_Dawn:\_The\_Series`).

Replace each `...` below with the resolved entity (NOTHING else inside the tags --- just the canonical name):

```
<entity_A>...</entity_A> (resolves {mask_a})
<entity_B>...</entity_B> (resolves {mask_b})
<entity_C>...</entity_C> (resolves {mask_c})
<entity_D>...</entity_D> (resolves {mask_d})
<entity_E>...</entity_E> (resolves {mask_e})
```

The answer to the question is whatever you put in <entity_A>.

{mask_list} is the five masks bulleted with “ - ”.

Multi-solution rewrite. The block from “Replace each ‘...’ below...” through “...in <entity_A>” is replaced with:

EUREQA: multi-solution rewrite

Provide {m} different reasoning chains, each wrapped in numbered tags <response_1>...</response_1> through <response_{m}>...</response_{m}>. Each chain must contain all 5 entity resolutions; replace each `...` with ONLY the resolved canonical name (underscored Wikipedia-style, no prose, no labels):

```
<response_i>
  <entity_A>...</entity_A> (resolves Person A --- this is the answer)
  <entity_B>...</entity_B> (resolves the second mask)
  <entity_C>...</entity_C> (resolves the third mask)
  <entity_D>...</entity_D> (resolves the fourth mask)
  <entity_E>...</entity_E> (resolves the fifth mask)
</response_i>
```

Closing tags are required for every response and every entity. The {m} chains should be genuinely different attempts (e.g. different candidate resolutions, not paraphrases). The answer to the question is whatever you put in <entity_A> within each chain.

E.3 Maze

The example below shows the prompt for one specific maze (the per-maze grid, item counts, and step budget are filled in per example).

Maze: single-solution prompt (GRPO, MaxRL, GDPO, MaxAtK, RandomW)

Navigate a 9x9 maze from S to E. Collect gold and diamonds, avoid lava.

Grid:

```

S . . . . . D .
# . # . . . D D
. . # # . . . D
. # L . . . L # .
. . . . B . . . .
. . . . . L . . .
G . . . . . .
G G . . . . .
G G . . . . # . E

```

- Move: UP, DOWN, LEFT, RIGHT. # is a wall -- you cannot enter it.
- Do not leave the grid.
- Collect: G (Gold), D (Diamond), B (Bonus) tiles by stepping on them.
- Avoid: L (Lava) tiles. Stepping on lava costs you.
- Visiting a B cell multiplies your other scores -- explore!
- You MUST reach E. If you don't reach E, your score is zero everywhere.
- Items only count if collected BEFORE you reach E (the trajectory ends at E).
- You have 27 steps.

This maze has 5 Gold, 4 Diamond, 3 Lava, and 1 Bonus tiles.
Output moves in <answer>...</answer> tags, e.g., <answer>UP UP RIGHT</answer>.

Multi-solution rewrite. The multi-solution prompt (used by VPO and Multi-RLVR with $m=3$) reuses the preamble, grid, and bullet list verbatim, except that “You have 27 steps.” becomes “You have 27 steps per route.”. The closing two lines (the tile-count line and the “Output moves ...” instruction) are replaced by the block below.

Maze: multi-solution rewrite ($m = 3$)

This maze has 5 Gold, 4 Diamond, 3 Lava, and 1 Bonus tiles.
Reason briefly about the maze, then provide 3 genuinely different routes from S to E.
Each route is a sequence of UP/DOWN/LEFT/RIGHT moves (space-separated).
Wrap each route in numbered tags (<route_1>...</route_1>, <route_2>...</route_2>, <route_3>...</route_3>). Inside each tag put ONLY moves (no arrows, no coordinates, no prose); any reasoning goes outside the tags. Each route has its own 14-step budget and must reach E (score is zero if it doesn't).
Format example ($m=3$):

```

<route_1>RIGHT RIGHT RIGHT RIGHT DOWN DOWN DOWN DOWN</route_1>
<route_2>DOWN DOWN DOWN DOWN RIGHT RIGHT RIGHT RIGHT</route_2>
<route_3>RIGHT DOWN RIGHT DOWN RIGHT DOWN RIGHT DOWN</route_3>

```

For general m , the tag list extends to <route_1>...</route_1>, ..., <route_m>...</route_m>.

E.4 ToolRL

ToolRL prompts come directly from the upstream qiancheng0/ToolRL r1la_4k corpus: the system message specifies the output format and the user message contains the dialogue task. We do not override the system prompt.

ToolRL: system message

You are a helpful multi-turn dialogue assistant capable of leveraging tool calls to solve user tasks and provide structured chat responses.

****Available Tools****
In your response, you can use the following tools:

1. Name: <tool_1_name>
Description: <...>
Parameters: {...}
2. Name: <tool_2_name>
- ...

```

**Steps for Each Turn**
1. **Think:** Recall relevant context and analyze the current user goal.
2. **Decide on Tool Usage:** If a tool is needed, specify the tool and its parameters.
3. **Respond Appropriately:** If a response is needed, generate one while maintaining consistency across user queries.

**Output Format**
<think> Your thoughts and reasoning </think>
<tool_call>
{"name": "Tool name", "parameters": {"Parameter name": "Parameter content", "... ..": "... .."}}
{"name": "... ..", "parameters": {"... ..": "... ..", "... ..": "... .."}}
...
</tool_call>
<response> AI's final response </response>

**Important Notes**
1. You must always include the `<think>` field to outline your reasoning. Provide at least one of `<tool_call>` or `<response>`. Decide whether to use `<tool_call>` (possibly multiple times), `<response>`, or both.
2. You can invoke multiple tool calls simultaneously in the `<tool_call>` fields. Each tool call should be a JSON object with a "name" field and an "parameters" field containing a dictionary of parameters. If no parameters are needed, leave the "parameters" field an empty dictionary.
3. Refer to the previous dialogue records in the history, including the user's queries, previous `<tool_call>`, `<response>`, and any tool feedback noted as `<obs>` (if exists).

```

The user message is the task itself, e.g.:

ToolRL: example user message

```

**Dialogue Records History**
<user> I need to make sure my library visit is smooth. Could you check if I'm a member, authorize my entry, log the access, and also verify the access control settings at the Main Library for the afternoon? </user>

```

Multi-solution rewrite. The system message is unchanged. The block below is appended to the *user* message.

ToolRL: multi-solution rewrite

```

---
ADDITIONAL INSTRUCTION (overrides the single-attempt output format above): Provide {m} different attempts at the task. Wrap each attempt in numbered outer tags <response_1>...</response_1> through <response_{m}>...</response_{m}>. Inside each <response_i>, follow the original output format from the system prompt (your <think>, optional <tool_call>, and inner <response> sections). The {m} attempts should be genuinely different --- different tool choices, different argument values, or different reasoning --- not paraphrases. Closing tags are required on every outer attempt.

```

E.5 Goal-Conditioning Suffix (Goal-Conditioned GRPO baseline only)

For the goal-conditioned GRPO baseline (any domain), the following block is *appended* to whichever variant is in use (single or multi). Per training example, weights are resampled from $\text{Dir}(1, \dots, 1)$.

Goal-conditioning suffix

Your objective weights (higher = more important to optimize):

```
- {desc_1}: {w_1:.2f}
- {desc_2}: {w_2:.2f}
...
```

Maximize your weighted score.

The per-domain `{desc_i}` strings are domain-specific component names; for MuSiQue, e.g., “Finding evidence for reasoning step 1”, ..., “Finding evidence for reasoning step 4”, “Answer word-level accuracy”.

F Reward Collinearity Predicts When VPO Helps

In §5 we argued that VPO’s gain over scalar baselines depends on whether the reward components are genuinely competing or effectively redundant. To make this prediction quantitative, we run a single cross-domain diagnostic: for each (domain, method) pair we measure (i) the on-policy collinearity of the reward vector under the trained model and (ii) the realized best@16 under the GRPO training scalar. The prediction is that VPO outperforms scalar GRPO precisely when on-policy reward dimensions are not collinear; UltraFeedback should be the only domain in which the prediction inverts.

Table 9 confirms this. Across Maze, MuSiQue, EUREQA, and ToolRL, on-policy $\bar{\rho}$ stays well below 1 and VPO wins best@16. On UltraFeedback the on-policy distribution is near-collinear ($\bar{\rho}_{\text{VPO}} = 0.95$, $\bar{\rho}_{\text{GRPO}} = 0.82$): the simplex collapses to a near-line, there is essentially one Pareto-optimal candidate per prompt, and VPO roughly matches GRPO. This is a sanity check on the regime claim in §5: when the reward is effectively scalar, VPO does not (and should not) outperform scalar GRPO.

Table 8: **Best@ k on UltraFeedback** under the ArmoRM-5 reward model. The on-policy reward simplex is near-collinear, so VPO loses to scalar GRPO on absolute best@ k while still retaining the largest per- k headroom of any method.

Method	best@1	best@5	best@10	best@20	best@50	Δ
Qwen3-4B (single-answer prompt)	0.700	0.724	0.730	0.735	0.741	+0.041
↪ GRPO	0.741	0.762	0.768	0.773	0.779	+0.038
↪ GDPO	0.737	0.759	0.765	0.770	0.776	+0.039
↪ Goal-cond	0.727	0.750	0.756	0.762	0.767	+0.040
Qwen3-4B (multi-answer prompt)	0.700	0.724	0.730	0.735	0.741	+0.041
↪ Multi-RLVR	0.730	0.763	0.770	0.775	0.780	+0.050
↪ VPO (ours)	0.700	0.755	0.763	0.769	0.776	+0.076

Table 9: **On-policy reward collinearity $\bar{\rho}$ vs. realized best@16** across all five domains. VPO’s advantage over scalar GRPO disappears exactly on UltraFeedback, the only domain where the on-policy components are near-collinear.

Domain	reward dims	on-policy $\bar{\rho}$		best@16		Δ
		VPO	GRPO	VPO	GRPO	
Maze	4 reward feats ($D=4$)	0.39	0.37	0.593	0.432	+0.161
MuSiQue	4 hop + 1 F1 ($D=5$)	0.12	0.11	0.864	0.841	+0.023
EUREQA	5 entity-EM ($D=5$)	0.05	0.03	0.204	0.182	+0.022
ToolRL	4 sub-metrics ($D=4$)	0.86	0.62	0.953	0.924	+0.028
UltraFeedback	5 ArmoRM dims ($D=5$)	0.95	0.82	0.767	0.772	-0.004

Methodology. For each (domain, method) pair we proceed as follows.

1. **On-policy rollout pool.** Sample the trained checkpoint for that method on every prompt in the held-out evaluation set, drawing N samples per prompt under the domain’s final-evaluation sampler (App. B). Per-domain pool shapes (prompts $\times$ samples $\times$ reward dims) are: Maze $500 \times 30 \times 4$ ([`completion`, `gold`, `diamond`, `avoid_lava`]); MuSiQue $300 \times 30 \times 5$ (4 hop indicators + answer-F1); EUREQA $682 \times 50 \times 5$ (entity-EM); ToolRL $80 \times N \times 4$; and UltraFeedback (epoch 2) $200 \times 50 \times 5$ (ArmoRM dims).
2. $\bar{\rho}$ (**on-policy off-diagonal Pearson correlation**). Flatten the rollout tensor to $(N_{\text{responses}}, D)$, drop zero-variance dimensions, compute the $D \times D$ Pearson correlation matrix (`np.corrcoef`), and average the $D(D-1)$ off-diagonal entries. We report $\bar{\rho}_{\text{VPO}}$ and $\bar{\rho}_{\text{GRPO}}$ on the same prompt set under each domain’s final-evaluation sampler.
3. **Best@16.** Scalarize each response with the GRPO training scalar for that domain (per-domain formulas in Table 7; the ArmoRM “overall” dim for UltraFeedback). For each prompt, evaluate the unbiased order-statistic estimator of $\mathbb{E}[\max_{k \in S} r_k]$ over a uniform 16-element subset S of the per-prompt sample pool, then average over prompts.